CSC 4161

AWS Lambda

Adrian Valenzuela
10/27/2026



Confidential

Copyright ©

Presentation Structure

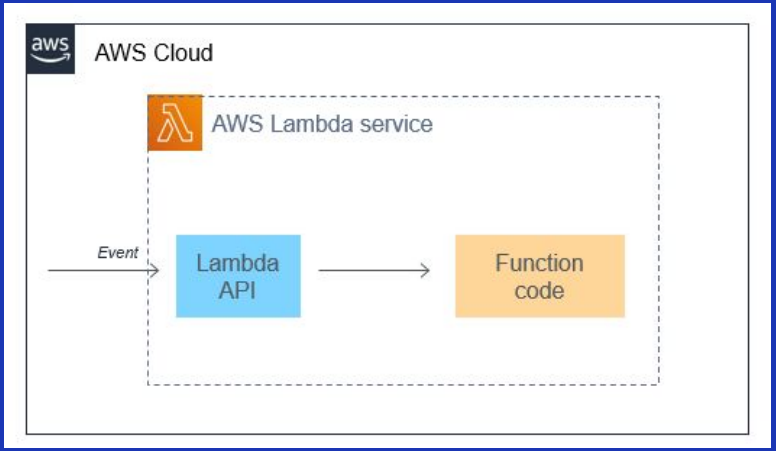
1. Mental Model
2. Where It Fits
3. Concrete Example
4. Personal Applications
5. Common Misconceptions
6. Wrap-Up

Confidential

Copyright ©

Mental Model

Instead of keeping a backend process running, invoke code when work arrives...
You manage: business logic, security permissions, tests, or error handling
AWS manages: Execution Infra, capacity provisioning, scaling



Source → **Trigger** → **Compute** → **Result**

Event
HTTP request, SQS message, file upload

Invoke
AWS routes the event to the function

Handler Runs
Your code (Java, Python, etc.) executes

Action
Return data, write a record, send an alert

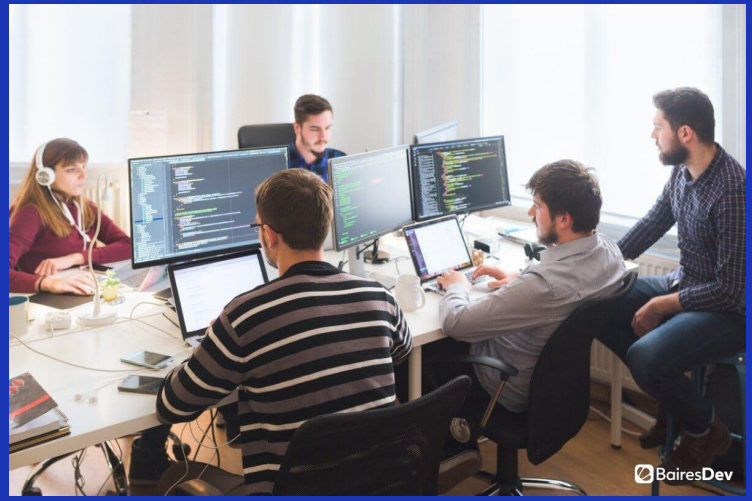
Confidential

Copyright ©

Where It Fits

Lambda works especially well when computation is triggered by a discrete event

- API Backends
- Message Queues
- File Processing
- Scheduled Jobs



Confidential

Copyright ©

What does a Lambda Function Look Like?

GreetingHandler.java — JAVA · EXAMPLE

```java
import com.amazonaws.services.lambda.runtime.*;
import java.util.Map;

public class GreetingHandler implements
    RequestHandler<Map<String, String>, String> {
  public String handleRequest(
      Map<String, String> event, Context context) {
    String name = event.get("name");
    return "Hello, " + name + "!";
  }
}
```

Input Event (JSON):

{ “name” : “Cal Poly” }

Returned Value:

“Hello, Cal Poly!”

Some important side notes: handleRequest is the entry point. Lambda calls it with event data; your code handles the logic

Confidential

Copyright ©

Experiences with Lambda

As an intern:

- Worked with AWS services including Lambda, SQS, DynamoDB, and CloudWatch.
- Developed a Replay API to reprocess failed messages from Amazon SQS dead-letter queues (DLQs).
- Worked with event-driven backend infrastructure, where services communicate asynchronously.

Here at Cal Poly (CSC 5100):

- Developed DominantAffectLambda, a Java-based AWS Lambda function for processing emotional state data.
- Implemented a Lambda handler to receive events, process incoming data, and return results.
- Integrated the function into an AWS cloud workflow.
- Used GitHub Actions and CI/CD workflows to test and deploy application code.

Confidential

Copyright ©

Common Misconceptions



Misconception #1: "Serverless means there are no servers."

- Servers still exist, but AWS manages the infrastructure.
- Developers focus primarily on application logic rather than server maintenance.

Misconception #2: "AWS Lambda is always the best choice."

- Lambda is best suited for event-driven and relatively short-lived workloads.
- Long-running applications or workloads requiring specialized infrastructure may benefit from other services.

Misconception #3: "AWS Lambda automatically handles everything."

- AWS handles infrastructure scaling and execution environments.
- Developers are still responsible for error handling, application security, monitoring, and reliable processing.

Conclusion and References

Key Takeaways

- AWS Lambda is a serverless computing service that executes code in response to events.
- It integrates with other AWS services, making it useful for APIs, automation, and asynchronous processing.
- It reduces infrastructure management but does not eliminate software engineering responsibilities.
- Choosing Lambda depends on the application's workload, performance requirements, and architecture.

Final Thought:

AWS Lambda allows developers to focus more on writing application logic and less on managing infrastructure, making it a valuable tool for building modern, event-driven applications.

References and Tools

- AWS Lambda Developer Guide: https://docs.aws.amazon.com/lambda/latest/dg/
- AWS Lambda Java Handlers: https://docs.aws.amazon.com/lambda/latest/dg/java-handler.html
- AWS Lambda with Amazon SQS: https://docs.aws.amazon.com/lambda/latest/dg/with-sqs.html